

Обучение с подкреплением: от основ до PER

Теоретическое введение для понимания работы

«Зависимость качества DQN-агента self-play от параметра приоритизации α в игре
«Уголки»»

Содержание

1	Зачем RL — и чем он отличается от обычного ML	3
2	Марковский процесс принятия решений (MDP)	3
2.1	Как это выглядит на практике	3
2.2	Пример из нашей работы	4
3	Политика и функции ценности	4
3.1	Политика	4
3.2	Суммарная дисконтированная отдача	4
3.3	Функция ценности состояния	4
3.4	Функция ценности действия (Q-функция)	5
3.5	Оптимальные функции ценности	5
4	Уравнения Беллмана	5
4.1	Уравнение Беллмана для Q^π	5
4.2	Оптимальное уравнение Беллмана	5
5	Табличное Q-обучение	6
5.1	TD-ошибка	6
5.2	Сходимость и ограничения	6
6	Deep Q-Network (DQN)	6
6.1	Нейросеть вместо таблицы	6
6.2	Проблема 1: корреляция в онлайн-обучении	7
6.3	Проблема 2: нестабильность TD-цели	7
6.4	Проблема 3: исследование среды	8
6.5	Маскировка нелегальных действий	8
6.6	Итоговый алгоритм DQN	8
7	Self-Play	8
7.1	Идея	8
7.2	Одна сеть для обоих игроков	9
7.3	Negamax bootstrapping	9
7.4	Нестационарность	9
8	Prioritized Experience Replay (PER)	9
8.1	Мотивация	9

8.2	Формула приоритетов	10
8.3	Параметр α — главный предмет исследования	10
8.4	Проблема смещения (bias) и IS-коррекция	11
8.5	Параметр β — степень коррекции	11
8.6	Полный алгоритм PER	11
9	Всё вместе: что и зачем в нашей работе	11
9.1	Полный pipeline	11
9.2	Что именно исследуется	12
9.3	Гипотеза о нестационарности	12
9.4	Метрика и оценка	12

1 Зачем RL — и чем он отличается от обычного ML

В supervised learning у нас есть датасет $\{(x_i, y_i)\}$: вход и правильный ответ. Задача — научить модель отображению $x \mapsto y$.

В обучении с подкреплением **правильного ответа нет**. Есть **агент**, который совершает действия в **среде**, и среда в ответ даёт **скалярную награду** (reward). Цель агента — выработать стратегию поведения, которая максимизирует суммарную награду.

Интуиция. Аналогия из жизни: обучение вождению. Инструктор не говорит «сейчас поверни руль на 23 градуса». Он говорит «хорошо» или «плохо», и вы постепенно сами нащупываете правильное поведение. RL — это примерно то же самое.

Ключевые отличия от supervised learning:

1. **Нет разметки.** Агент не получает «правильное действие» — он получает только награду, которая может прийти много ходов спустя.
2. **Отложенное вознаграждение.** Действие сейчас влияет на ситуацию через много шагов. Это называется *temporal credit assignment problem*.
3. **Распределение данных зависит от агента.** В отличие от фиксированного датасета, какие ситуации мы будем видеть — зависит от того, как агент себя ведёт.

2 Марковский процесс принятия решений (MDP)

Формальная математическая основа RL — **Марковский процесс принятия решений** (Markov Decision Process, MDP).

MDP. MDP задаётся кортежем $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, где:

- $\mathcal{S}$ — пространство состояний,
- $\mathcal{A}$ — пространство действий,
- $P(s' | s, a)$ — функция переходов: вероятность оказаться в состоянии s' , если в состоянии s выбрать действие a ,
- $R(s, a, s')$ — немедленная награда,
- $\gamma \in [0, 1)$ — коэффициент дисконтирования.

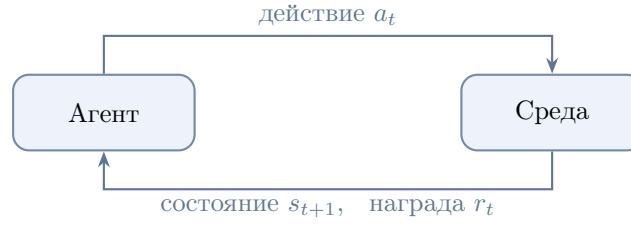
Марковское свойство означает, что следующее состояние s' зависит только от текущего состояния s и действия a , но не от всей предыстории. Это принципиально: мы можем строить оптимальные стратегии, смотря только на текущее состояние.

2.1 Как это выглядит на практике

На каждом шаге t :

1. Агент наблюдает состояние s_t .
2. Агент выбирает действие a_t .
3. Среда переходит в новое состояние $s_{t+1} \sim P(\cdot | s_t, a_t)$.

4. Агент получает награду $r_t = R(s_t, a_t, s_{t+1})$.



2.2 Пример из нашей работы

В игре «Уголки»:

- $\mathcal{S}$ — все возможные позиции фишек на доске 8×8 ; закодировано как тензор $3 \times 8 \times 8$.
- $\mathcal{A}$ — все пары клеток (откуда, куда): $64 \times 64 = 4096$ действий. Большинство нелегальны.
- P — детерминированный (игра без случайности).
- R — $+100$ за победу, -100 за поражение, $+0,2 \times \Delta d$ за приближение к цели, $-0,01$ за каждый ход.
- $\gamma = 0,99$.

3 Политика и функции ценности

3.1 Политика

Политика. Политика π — это стратегия агента. Стохастическая политика задаётся распределением $\pi(a | s)$ — вероятностью выбрать действие a в состоянии s . Детерминированная политика задаётся функцией $a = \pi(s)$.

3.2 Суммарная дисконтированная отдача

После момента времени t агент получит следующие награды: $r_t, r_{t+1}, r_{t+2}, \dots$. Мы хотим максимизировать не одну награду, а всю **будущую отдачу**:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \quad (1)$$

Зачем нужен $\gamma < 1$? Во-первых, математически: сумма сходится. Во-вторых, содержательно: награда сейчас ценнее, чем та же награда потом (как в экономике). Типичное значение $\gamma = 0,99$ означает, что награда через 100 шагов стоит $0,99^{100} \approx 0,37$ от её номинала — это «горизонт» примерно 100 шагов.

3.3 Функция ценности состояния

Функция V^π . Ценность состояния s при политике π — ожидаемая будущая отдача, если начать из s и следовать π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

V^π отвечает на вопрос: «Насколько хорошо оказаться в состоянии s , если дальше следовать политике π ?»

3.4 Функция ценности действия (Q-функция)

Q-функция. Ценность действия a в состоянии s при политике π :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$$

$Q^\pi(s, a)$ отвечает на вопрос: «Насколько хорошо выбрать именно a в s , а потом следовать π ?»

Связь между ними:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a)$$

3.5 Оптимальные функции ценности

Нас интересует **оптимальная** политика π^* — та, которая максимизирует $V^\pi(s)$ для всех состояний одновременно. Оптимальные функции ценности:

$$V^*(s) = \max_\pi V^\pi(s), \quad Q^*(s, a) = \max_\pi Q^\pi(s, a)$$

Связь между ними фундаментальна: если мы знаем Q^* , оптимальная политика тривиальна:

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

Именно поэтому Q-обучение так популярно: достаточно научить сеть приближать Q^* .

4 Уравнения Беллмана

Уравнения Беллмана — это рекуррентные соотношения, которые связывают ценность текущего состояния с ценностями следующих. Они лежат в основе всех алгоритмов RL.

4.1 Уравнение Беллмана для Q^π

Раскроем определение Q^π через определение G_t :

$$Q^\pi(s, a) = \mathbb{E}[r + \gamma Q^\pi(s', \pi(s')) \mid s, a] \quad (2)$$

Иными словами, ценность $Q^\pi(s, a)$ равна немедленной награде плюс дисконтированная ценность в следующем состоянии при продолжении следования π .

4.2 Оптимальное уравнение Беллмана

Для оптимальной Q^* :

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a') \mid s, a] \quad (3)$$

Это уравнение Беллмана оптимальности. Оно самосогласованное: Q^* и есть решение этого уравнения. Именно к нему будет сходиться Q-обучение.

Интуиция. Аналогия с ML. В supervised learning у нас есть функция потерь $\mathcal{L}(\theta) = \mathbb{E}[(y - f_\theta(x))^2]$ и мы ищем θ^* минимизирующий её. В RL задача хитрее: «правильный ответ» $r + \gamma \max_{a'} Q^*(s', a')$ сам зависит от Q^* , которую мы ищем. Поэтому нужны специальные методы.

5 Табличное Q-обучение

Самый простой способ найти Q^* — хранить все значения $Q(s, a)$ в большой таблице и постепенно обновлять их.

5.1 TD-ошибка

На каждом шаге мы наблюдаем переход (s, a, r, s') и вычисляем **ошибку временной разности** (temporal difference error, TD-ошибка):

$$\delta = \underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{TD-цель}} - \underbrace{Q(s, a)}_{\text{текущая оценка}} \quad (4)$$

Затем обновляем таблицу:

$$Q(s, a) \leftarrow Q(s, a) + \alpha_{\text{lr}} \delta$$

где α_{lr} — шаг обучения (не путать с параметром PER α !).

Интуиция. Аналогия с SGD. TD-обновление — это как один шаг градиентного спуска по функции потерь $\mathcal{L} = \delta^2/2$, только «правильный ответ» (TD-цель) временно фиксируется. Это и есть суть bootstrapping в RL.

5.2 Сходимость и ограничения

Алгоритм Q-обучения был предложен Уоткинсом в его докторской диссертации (1989) и формально проанализирован совместно с Дзяном в работе [1]. Авторы доказали, что при двух условиях — каждая пара (s, a) посещается бесконечно часто и шаг обучения α_{lr} убывает по правилу Робинса–Монро — Q-обучение сходится к Q^* с вероятностью 1. Это фундаментальный теоретический результат, на котором стоит весь Q-learning и его потомки.

Проблема: если $|\mathcal{S}| \times |\mathcal{A}|$ велико (в нашем случае: $\sim 10^{30} \times 4096$), таблица не помещается в память и условие «посетить каждую пару бесконечно часто» невыполнимо на практике. Нужна аппроксимация функции Q .

6 Deep Q-Network (DQN)

6.1 Нейросеть вместо таблицы

Прорыв совершила работа Мниха и соавторов из DeepMind [2], опубликованная в *Nature* в 2015 году. Авторы показали, что **одна нейронная сеть** способна играть в 49 различных видеоигр Atari на уровне человека или выше, получая на вход только пиксели экрана и счёт — без какого-либо специфичного для игры знания. До этого считалось, что каждая задача требует своей ручной настройки. Ключевых нововведений было два: Experience Replay и Target Network — оба описаны ниже.

Идея DQN [2]: заменить таблицу нейронной сетью $Q_\theta(s, a)$ с параметрами θ . Функция потерь:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\underbrace{\left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_\theta(s, a) \right)^2}_{y - \text{TD-цель}} \right] \quad (5)$$

Обратите внимание на θ^- в TD-цели — об этом ниже.

В нашей работе:

$$Q_\theta : \mathbb{R}^{3 \times 8 \times 8} \rightarrow \mathbb{R}^{4096}$$

Сеть принимает на вход тензор состояния и выдаёт **сразу все** Q -значения для всех 4096 действий за один прямой проход.

6.2 Проблема 1: корреляция в онлайн-обучении

Если обучать сеть сразу на каждом переходе (s_t, a_t, r_t, s_{t+1}) в порядке их получения, данные будут сильно скоррелированы: соседние состояния похожи, градиенты «тянут» сеть в одном направлении, и обучение нестабильно.

Решение: Experience Replay [4].

Идея буфера опыта была предложена Линем ещё в 1992 году в контексте обучения роботов: вместо того чтобы выбрасывать каждый переход после одного использования, его сохраняют и многократно «переигрывают». Лин показал, что это существенно повышает эффективность использования данных в нескольких задачах управления. Мних и соавторы [2] заново открыли и популяризировали эту идею применительно к глубоким нейросетям, превратив её в стандартный компонент DQN-алгоритмов.

Буфер опыта. Все переходы $(s, a, r, s', \text{done})$ записываются в кольцевой буфер $\mathcal{D}$ размером C (у нас $C = 10^5$). На каждом шаге из буфера случайно сэмпляется мини-батч размером B и делается один шаг градиентного спуска.

Это аналог перемешивания датасета в supervised learning: разрываем корреляции, получаем i.i.d.-подобные батчи.

Интуиция. Буфер работает как «память» агента. Старые переходы постепенно вытесняются новыми по мере заполнения. Размер буфера — компромисс: маленький буфер означает мало разнообразия, слишком большой — много устаревших данных от старой политики.

6.3 Проблема 2: нестабильность TD-цели

Если TD-цель $y = r + \gamma \max_{a'} Q_\theta(s', a')$ вычисляется той же сетью θ , которую мы обновляем, то цель «убегает» во время обучения — мы гонимся за постоянно движущейся мишенью. Это приводит к расходимости.

Решение: Target Network (целевая сеть) [2]. Это второе ключевое нововведение оригинальной статьи DeepMind.

Target Network. Ведём две копии сети: **online**-сеть Q_θ (обучается каждый шаг) и **target**-сеть Q_{θ^-} (заморожена). TD-цель считается по Q_{θ^-} . Каждые T_{target} шагов делаем жёсткое копирование: $\theta^- \leftarrow \theta$.

У нас $T_{\text{target}} = 1000$ шагов.

6.4 Проблема 3: исследование среды

Если агент всегда выбирает действие с максимальным Q , он может застрять в локальном оптимуме. Нужен баланс между *exploitation* (использовать знания) и *exploration* (исследовать неизвестное).

Решение: ϵ -жадная политика.

ϵ -жадная политика.

$$a_t = \begin{cases} \text{случайное легальное действие} & \text{с вероятностью } \epsilon \\ \arg \max_a Q_\theta(s_t, a) & \text{с вероятностью } 1 - \epsilon \end{cases}$$

ϵ линейно убывает от 1,0 до 0,05 за 50 000 шагов: сначала почти чистое исследование, потом почти чистая эксплуатация.

6.5 Маскировка нелегальных действий

В нашей задаче многие из 4096 действий нелегальны (фишки нет на клетке, нельзя двигаться так). Перед выбором действия нелегальные маскируются:

$$Q_\theta(s, a) \leftarrow -\infty \quad \text{для всех нелегальных } a$$

Тогда $\arg \max$ гарантированно вернёт легальное действие.

6.6 Итоговый алгоритм DQN

1. Инициализировать θ , $\theta^- \leftarrow \theta$, буфер $\mathcal{D}$.
2. Для каждого шага:
 - (a) Наблюдать s_t , выбрать a_t по ϵ -жадной политике.
 - (b) Сделать ход, получить r_t, s_{t+1} , записать в $\mathcal{D}$.
 - (c) Сэмплировать батч из $\mathcal{D}$.
 - (d) Вычислить TD-цели: $y_i = r_i + \gamma \max_{a'} Q_{\theta^-}(s'_i, a')$.
 - (e) Сделать шаг градиентного спуска: $\theta \leftarrow \theta - \nabla_\theta \frac{1}{B} \sum_i (y_i - Q_\theta(s_i, a_i))^2$.
 - (f) Обновить θ^- каждые T_{target} шагов.

7 Self-Play

7.1 Идея

В однопользовательских задачах RL (например, Atari) есть агент и фиксированная среда. В играх с нулевой суммой (как «Уголки») у нас два игрока. Self-play — это режим, в котором **агент играет сам с собой**: обе стороны используют одну и ту же сеть.

Это изящное решение: по мере того как сеть улучшается, соперник тоже становится сильнее, создавая постоянное давление на дальнейший рост.

Исторически self-play хорошо зарекомендовал себя в нардах (TD-Gammon, Tesauro, 1994) и достиг пика известности в серии работ AlphaGo/AlphaZero компании DeepMind: в 2017 году AlphaZero обучилась игре в шахматы, сёги и го **только через self-play**, не используя ни одной партии человека, и превзошла все специализированные программы. Мы адаптируем ту же идею к «Уголкам»: вместо огромной вычислительной инфраструктуры AlphaZero — лёгкий DQN, который, тем не менее, успешно осваивает игровую стратегию.

7.2 Одна сеть для обоих игроков

Ключевая техника — **канонический фрейм**. Доска всегда кодируется с точки зрения текущего игрока: его фишки — на своей позиции, чужие — на чужой. Для Игрока -1 доска поворачивается на 180.

Таким образом, сеть всегда видит одно и то же: «свои фишки сверху-слева, надо перенести их вниз-вправо» — независимо от того, кто ходит.

7.3 Negamax bootstrapping

В обычном DQN TD-цель:

$$y = r + \gamma \max_{a'} Q_{\theta-}(s', a')$$

Но в self-play после хода Игрока 1 ход переходит к Игроку -1 . Состояние s' кодируется уже *с точки зрения Игрока -1* . А игра нулевая сумма, значит:

$$V(s \text{ для Игрока } 1) = -V(s \text{ для Игрока } -1)$$

Поэтому TD-цель в self-play корректируется знаком:

$$y = r - \gamma \max_{a'} Q_{\theta-}(s', a') \quad (6)$$

Это называется **negamax bootstrapping**. Без этой поправки цели были бы неверными и сеть обучалась бы «помогать» противнику.

7.4 Нестационарность

В обычном RL среда фиксирована — $P(s'|s, a)$ не меняется. В self-play «соперник» — это та же сеть, которая обновляется. С точки зрения каждого из игроков среда **нестационарна**: распределение P меняется вместе с параметрами θ .

Это важно для понимания того, почему гиперпараметры из статических сред (например, Atari) могут не работать в self-play.

8 Prioritized Experience Replay (PER)

8.1 Мотивация

Метод Prioritized Experience Replay был предложен Шаулем и соавторами из DeepMind [3] и представлен на ICLR 2016. Авторы показали, что на **41 из 49 игр Atari** агент с PER опережает стандартный DQN с равномерным буфером при одинаковом числе шагов обучения. В оригинальной работе в качестве рабочего значения параметра приоритизации выбрано $\alpha = 0,6$ — оно оказалось лучшим компромиссом между полной приоритизацией и равномерностью именно для статичных сред Atari. Вопрос о том, сохраняется ли этот результат в нестационарных средах, таких как self-play, и является предметом нашей работы.

В стандартном Experience Replay все переходы сэмпляются равномерно. Но не все переходы одинаково полезны:

- Переход с большой TD-ошибкой $|\delta|$ означает, что сеть сильно ошиблась в своей оценке — это «неожиданный» и потенциально информативный переход.
- Переход с малой $|\delta|$ означает, что сеть уже хорошо знает это место — повторное обучение на нём принесёт мало пользы.

Интуиция. Аналогия из обучения: если вы хорошо решаете задачи типа А и плохо типа В, разумно чаще практиковать В. PER делает то же самое — «сложные» переходы сэмпляются чаще.

8.2 Формула приоритетов

Приоритеты в PER [3]. Каждому переходу i в буфере назначается приоритет:

$$p_i = |\delta_i| + \varepsilon$$

где $\varepsilon = 10^{-6}$ — маленькая константа, чтобы никакой переход не получил нулевой приоритет. Вероятность сэмпирования:

$$P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha}$$

8.3 Параметр α — главный предмет исследования

Параметр $\alpha \in [0, 1]$ контролирует **степень приоритизации**:

Значение α	Поведение
$\alpha = 0$	$P(i) = \text{const}$ — равномерная выборка (стандартный DQN)
$\alpha = 0,6$	умеренная приоритизация (стандарт для Atari)
$\alpha = 1$	$P(i) \propto \delta_i $ — полная приоритизация по TD-ошибке

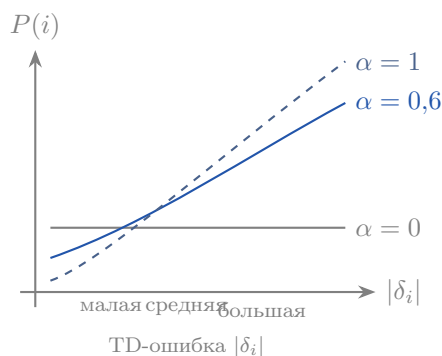


Рис. 1: Зависимость вероятности выборки от TD-ошибки при разных α . При $\alpha = 0$ все переходы равновероятны; при $\alpha = 1$ переходы с большой ошибкой доминируют.

8.4 Проблема смещения (bias) и IS-коррекция

Когда мы сэмплируем не равномерно, оценка градиента становится смещённой: переходы с высоким приоритетом «перегружают» функцию потерь. Чтобы это скомпенсировать, каждому переходу назначается **importance sampling (IS) вес**:

$$w_i = \frac{1}{N \cdot P(i)}^{-\beta} = (N \cdot P(i))^{-\beta} \quad (7)$$

Для численной стабильности нормируем на $\max_j w_j$:

$$\tilde{w}_i = \frac{w_i}{\max_j w_j}$$

Скорректированная функция потерь:

$$\mathcal{L}_{\text{PER}}(\theta) = \frac{1}{B} \sum_{i=1}^B \tilde{w}_i \cdot (y_i - Q_{\theta}(s_i, a_i))^2 \quad (8)$$

8.5 Параметр β — степень коррекции

$\beta \in [0, 1]$ контролирует степень IS-коррекции:

- $\beta = 0$: IS-веса все равны 1 — коррекции нет.
- $\beta = 1$: полная коррекция смещения.

На практике β **анилируется** от 0,4 до 1,0 по мере обучения. В начале обучения агрессивная коррекция мешает — сеть ещё нестабильна. К концу, когда Q -значения уже более точные, нужна полная коррекция для гарантий сходимости.

В нашей работе: $\beta : 0,4 \rightarrow 1,0$ за 500 000 шагов.

8.6 Полный алгоритм PER

По сравнению с обычным DQN добавляются три операции:

1. **При записи** нового перехода в буфер: назначить ему максимальный текущий приоритет (гарантирует, что новый переход будет отобран хотя бы раз).
2. **При сэмплировании**: сэмплировать пропорционально $P(i)$, вычислить IS-веса $\tilde{w}_i$.
3. **После gradient step**: пересчитать δ_i для отобранных переходов и **обновить** их приоритеты в буфере.

9 Всё вместе: что и зачем в нашей работе

Теперь можно понять каждый компонент.

9.1 Полный pipeline

1. **Среда** — ортогональные «Уголки» 8×8 . MDP с детерминированными переходами и reward shaping.
2. **Кодирование** — тензор $3 \times 8 \times 8$ в каноническом фрейме. Один тензор описывает состояние с точки зрения любого из двух игроков.

3. **DQN-сеть** — 2 свёрточных слоя + 2 линейных, выход — 4096 Q-значений. Нелегальные действия маскируются $-\infty$ перед $\arg \max$.
4. **Self-play** — оба игрока используют одну сеть. Negamax bootstrapping (6) обеспечивает правильный знак TD-цели в игре с нулевой суммой.
5. **Imitation warm-start** — перед self-play агент несколько итераций обучается поведенческому клонированию от эвристического агента. Это даёт разумную стартовую политику и ускоряет дальнейшее обучение.
6. **PER (или Uniform)** — буфер опыта 10^5 переходов. В варианте PER сэмплирование пропорционально $(|\delta_i| + \epsilon)^\alpha$; приоритеты обновляются после каждого gradient step.

9.2 Что именно исследуется

Работа ставит один вопрос: **как параметр α влияет на конечное качество политики** при фиксированных всех остальных гиперпараметрах?

Проверяются четыре значения: $\alpha \in \{0, 0,3, 0,6, 0,9\}$. $\alpha = 0$ соответствует стандартному DQN без приоритизации.

9.3 Гипотеза о нестационарности

Стандартное значение $\alpha = 0,6$ было подобрано для задач Atari, где среда фиксирована. В self-play «соперник» — это та же обновляемая сеть. Это означает:

- TD-ошибки прошлых переходов быстро устаревают: то, что было «трудным» для старой политики, может стать лёгким для новой.
- При высоком α такие устаревшие «трудные» переходы продолжают доминировать в выборке $\rightarrow$ перекошенный градиент.
- IS-коррекция (β) компенсирует смещение, но в начале обучения $\beta = 0,4$ и коррекция неполная.

Гипотеза: **агрессивная приоритизация дестабилизирует self-play**, приводя к пиковым TD-ошибкам и снижению качества политики.

Дополнительный аргумент в пользу осторожности при выборе α даёт работа Хесселя и соавторов [5], создавших **Rainbow** — комбинацию шести улучшений DQN одновременно (включая PER, Double DQN, Dueling и другие). Авторы обнаружили, что гиперпараметры компонентов *взаимодействуют*: оптимальное α в изоляции может оказаться неоптимальным в сочетании с другими методами. В нашем случае «дополнительным компонентом», меняющим характер обучения, выступает именно self-play.

Таким образом, литература даёт основания полагать, что $\alpha = 0,6$ не является универсально лучшим выбором и может требовать адаптации под конкретную среду.

9.4 Метрика и оценка

Агент оценивается в 1000 партиях против Greedy и Heuristic агентов. Метрика — **доля побед среди результативных партий**:

$$\text{win rate} = \frac{\text{победы}}{\text{победы} + \text{поражения}}$$

Ничьи исключены, потому что при лимите 300 ходов их очень много (57–74% против Random) — они заглушают сигнал.

Для статистики используется **bootstrap confidence interval**: из результатов n партий делается 10 000 ресэмплов с возвращением, по каждому считается win rate, берётся 2.5-й и 97.5-й перцентили. Это даёт 95% ДИ, не требуя нормальности распределения.

Краткий словарь

Термин	Пояснение
Agent / Агент	Обучаемая система, выбирающая действия.
Environment / Среда	Всё, с чем взаимодействует агент.
State s	Полное описание текущей ситуации.
Action a	Ход, который делает агент.
Reward r	Скалярный сигнал обратной связи от среды.
Return G_t	Дисконтированная сумма будущих наград (1).
γ	Коэффициент дисконтирования (у нас 0.99).
Policy π	Стратегия: как выбирать действие в состоянии.
$Q(s, a)$	Ожидаемая отдача при действии a в s .
TD-ошибка δ	Разность TD-цели и текущей оценки Q (4).
Experience Replay	Буфер переходов для разрыва корреляций.
Target Network	Замороженная копия сети для стабильных TD-целей.
ϵ -жадная	Политика: случайное действие с вероятностью ϵ .
Self-play	Агент играет против собственной копии.
Negamax	Знаковая поправка TD-цели для zero-sum игр (6).
PER	Prioritized Experience Replay — выборка по $ \delta $ [3].
α (PER)	Степень приоритизации. $\alpha = 0$ — равномерно.
β (PER)	Степень IS-коррекции. $\beta = 1$ — полная коррекция.

Список литературы

- [1] Watkins C.J.C.H., Dayan P. Q-learning. *Machine Learning*, 8:279–292, 1992.
- [2] Mnih V. et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [3] Schaul T. et al. Prioritized Experience Replay. *ICLR*, 2016.
- [4] Lin L.-J. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–321, 1992.
- [5] Hessel M. et al. Rainbow: Combining improvements in deep reinforcement learning. *AAAI*, 2018.